

Lista de Exercícios – 01

Nome:

Arquivos Sequenciais

1. Prove que é necessário somente 1 acesso a registro para se encontrar qualquer chave utilizando busca por interpolação, se: a diferença entre os valores de quaisquer duas chaves consecutivas for constante e os valores da menor e da maior chave forem conhecidos.
2. Considere o arquivo:
Índices: 1 2 3 4 5 6 7 8 9 10
Chaves: 1 7 10 20 21 30 35 40 45 61
 - (a) Apresente as comparações realizadas pela busca por interpolação na busca pela chave 21.
 - (b) Cite um exemplo de chave para o qual a busca binária faria o maior número de comparações (pior caso). Quais comparações seriam feitas?
3. Com base no mesmo arquivo:
 - (a) Apresente as comparações feitas pela busca por interpolação para encontrar a chave 35.
 - (b) Apresente as comparações feitas pela busca binária para encontrar a chave 35.
4. Qual é a complexidade da busca binária no melhor e pior caso? Dê exemplos de arquivos e chaves.
5. Qual é a complexidade da busca por interpolação no melhor e pior caso? Dê exemplos de arquivos e chaves.
6. Como expressar a complexidade da busca binária e da interpolação no caso de busca sem sucesso?
7. Qual seria o comportamento do Algoritmo 1 se a condição do laço fosse alterada de $a \leq b$ para $a < b$? Analise as consequências desta modificação para busca binária e para busca por interpolação. Se necessário, adapte o algoritmo e a função `nextPos`.

entrada : Arquivo ordenado A por chave primária com n registros; valor k a ser buscado
saida : Índice do registro no arquivo cuja chave é k (caso exista) ou -1 (caso contrário)

$a \leftarrow 1; b \leftarrow n;$

while $a \leq b$ **do**

$i \leftarrow \text{nextPos}(k, a, b);$

if $v = A[i].k$ **then**

return $i;$

else if $k > A[i].k$ **then**

$a \leftarrow i + 1;$

else

$b \leftarrow i - 1;$

return -1

/ deve garantir que $a \leq i \leq b$ */*

Algorithm 1: Busca por particionamento sucessivos

8. Qual o motivo de expressão abaixo estar sub-dividida em dois casos? Esta função `nextPos` poderia ser usada no Algoritmo 1 sem requerer outras modificações? Justifique.

$$\text{nextPos}(v, a, b) = \begin{cases} a & \text{se } a = b \\ \max \left\{ a, \min \left\{ b, \left\lceil a + \frac{(k - A[a].k)(b - a)}{A[b].k - A[a].k} \right\rceil \right\} \right\} & \text{se } a \neq b \end{cases}$$

9. Como busca por interpolação oferece vantagens sobre busca binária apenas quando os valores das chaves estão uniformemente distribuídos ao longo do arquivo, elabore um algoritmo que mescla ambas as estratégias, adaptando-se durante a busca. Compare experimentalmente sua solução, exibindo o comportamento semelhante às ilustrações feitas em sala, plotando número de acessos médio por tamanho do arquivo.
10. Analise o procedimento de busca por interpolação dado pelo Algoritmo 2, comparando-o com aquele apresentado pelo Algoritmo 1.
 - (a) Há alguma possibilidade de o denominador da linha 3 ser nulo? Por que?
 - (b) O algoritmo poderia ser modificado removendo a linha 7 e alterando a linha 4 para retornar o valor de i ? Discuta esta possibilidade, indicando esta modificação é ou não correta.

entrada : Arquivo ordenado por chave primária com n registros; valor v a ser buscado
saída : Índice do registro no arquivo cuja chave é k (caso exista) ou -1 (caso contrário)

```

 $a \leftarrow 1; b \leftarrow n;$ 
while ( $A[a].k < v \leq A[b].k$ ) do
   $i \leftarrow \left\lceil i + \frac{(k - A[a].k)(b - a)}{A[b].k - A[a].k} \right\rceil;$ 
  if  $k = A[i].k$  then  $a \leftarrow i;$ 
  else if  $k > A[i].k$  then  $i \leftarrow i + 1;$ 
  else  $b \leftarrow i - 1;$ 
if  $A[a].k = k$  then return  $i;$ 
return  $-1;$ 

```

Algorithm 2: Busca por Interpolação

11. Usando um exemplo com 10 registros, forneça o pior caso para:
 - (a) busca binária
 - (b) busca por interpolação
12. Encontre a expressão que fornece o número médio de acessos para recuperar um registro existente num arquivo com n registros usando busca binária. Considere que qualquer registro do arquivo tem igual probabilidade de ser buscado.
 - Dica: observe o número de acessos para cada registro; 1 registro é buscado com 1 acesso; 2 registros são buscados com 2 acessos; 4 registros são buscados com 3 acessos; e assim sucessivamente.
13. Derive a expressão que fornece o número médio de acessos do Algoritmo 1 caso a linha 1 fosse substituída por $\text{nextPos}(v, a, b) = \text{random}(a, b)$.
14. Compare experimentalmente o comportamento do Algoritmo 1 considerando que
 - (a) $\text{nextPos}(v, a, b) = \text{random}(a, b)$.
 - (b) $\text{nextPos}(v, a, b) = a + \lceil (a + b)/3 \rceil$.
 - (c) $\text{nextPos}(v, a, b) = a + \lceil (a + b)/10 \rceil$.
15. Similarmente ao que foi apresentado em sala, ilustre o comportamento de busca binária e busca por interpolação para tamanhos de arquivos $n \in [1000, 100000]$, considerando que os valores das chaves dos registros estão distribuídos de acordo com $\mathcal{N}(1000, 100)$, uma distribuição Normal com média $\mu = 1000$ e variância $\sigma^2 = 100$. Recomenda-se que haja ao menos 10 repetições do experimento para cada valor considerado de n . O gráfico a ser exibido deve adotar a média dos valores obtidos para cada valor de n .

Hashing

Para as questões a seguir, se a função hashing não for dada, considere:

- $h_1(chave) = chave \bmod 11$; e
- $h_2(chave) = \lfloor chave/11 \rfloor$, com valor mínimo 1.

Considere ainda que o método do encadeamento refere-se ao encadeamento explícito com alocação estática de memória.

1. Apresente o estado final do arquivo com encadeamento:
 - (a) após inserção das chaves: 26, 32, 15, 52, 20, 17, 18, 37, 12, 13;
 - (b) após a remoção da chave 15.

2. Repita a inserção da questão anterior utilizando o método da árvore binária para realocação de registros. Houve vantagem comparando ao encadeamento?
3. Repita a questão anterior considerando o método de Brent.
4. Repita as questões ??-?? para os registros 25, 31, 14, 51, 19, 16, 17, 36, 11, 12 e, em seguida, considerando a remoção da chave 16.
5. Repita as inserções dos exemplos das questões anteriores usando Linear Probing.
6. Insira as chaves: 39, 26, 15, 19, 42, 20, 18, 13, 27, 31 com encadeamento. Qual o número médio de acessos? O uso dos métodos de Brent e árvore binária melhorariam o número de acessos?
7. Dada a sequência: 16, 27, 10, 30, 18, 38, 32, 49, responda:
 - (a) Estado final do arquivo com encadeamento.
 - (b) Número médio de acessos.
 - (c) Estado após remoção da chave 16.
 - (d) Estado após remoção apenas da chave 27 (sem remover 16).
8. Insira a sequência anterior usando árvore binária.
9. Insira: 17, 28, 21, 42, 20, 39, 19, 30 usando:
 - (a) Encadeamento
 - (b) Linear Probing
 - (c) Hashing Duplo
 - (d) Árvore Binária
 - (e) Método de Brent
10. Calcule e compare o número médio de acessos para cada método acima.
11. Considere que a função $h(x, i) = (h_1(x) + i h_2(x)) \bmod m$ é usada para computar o endereço numa tabela *hashing* de tamanho $m = 7$ para uma chave de valor x . O valor de $i = 0, 1, \dots$ representa o número de sondagens necessárias para o endereço apropriado. Em caso de colisões, valores crescentes de i são usados. Por exemplo, $h(x, 0)$ fornece o endereço de x em apenas uma sondagem enquanto que $h(x, 1)$ corresponde a duas sondagens. Em outras palavras, $h_2(x)$ fornece o incremento usado durante a busca do endereço na tabela. Considere ainda que $h_1(x) = x \bmod m$ e $h_2(x) = 1 + (x \bmod (m - 1))$. Usando as funções mencionadas, inclua os registros com chaves $\{14, 28, 42, 56, 70, 84\}$, nesta ordem, numa tabela inicialmente vazia sem uso de ponteiros (endereço aberto).
 - (a) Mostre a configuração final da tabela
 - (b) Forneça o número médio de acessos para recuperar cada um dos registros
12. Considere as funções dadas na questão anterior, $m = 7$ e o método da árvore binária para realocação de registros. A partir dos dados ilustrados na tabela *hash* a seguir, inclua o registro com chave 21. Mostre a configuração final da tabela e a árvore formada durante a inclusão. Endereços indicados com ‘-’ estão desocupados.

endereço	0	1	2	3	4	5	6
chave	58	23	79	-	67	98	-

13. Considere a seguinte sequência de inteiros X , cujos valores representam chaves de registros.

$$X = (6, 90, 22, 18, 30, 39, 51, 69, 73, 33). \quad (1)$$

Considere ainda as seguintes funções *hashing*, com m , m_1 e m_2 inteiros positivos. Em $h(x, i)$, o parâmetro i , $i = 0, 1, 2, \dots$, deve ser interpretado como a i -ésima sondagem para a chave x .

$$h(x, i) = (h_1(x) + i h_2(x)) \bmod m \qquad h_1(x) = x \bmod m_1 \qquad h_2(x) = 1 + (x \bmod m_2)$$

Sabendo que h é usada numa implementação de *hashing* com tratamento de colisão por sondagem linear, forneça o maior valor possível, não superior a m , para:

- (a) m_1 ;
- (b) m_2 .

14. Considerando o método *hashing* duplo, $m = 11$ e a função h da questão anterior, responda:
- Inclua as chaves em X , da esquerda para a direita, e mostre a configuração final da tabela. Assuma que $m_1 = 7$ e $m_2 = 5$.
 - A escolha dos parâmetros m_1 e m_2 no item anterior foi adequada? Sim sim, justifique. Caso contrário, forneça valores adequados.
15. Considere a tabela *hashing* a seguir que ilustra como as chaves x_i estão dispostas nos endereços. As linhas ‘antes’ e ‘após’ mostram respectivamente as configurações da tabela antes e após a inserção do registro com chave x_5 . Endereços indicados com ‘-’ estão desocupados. Suponha que usou-se o método da árvore binária para relocação de registros. Responda:
- Qual o valor de $h(x_5, 0)$?
 - Qual valor de $h_2(x_0)$?
 - Forneça os possíveis valores para $h_2(x_5)$?

endereço	0	1	2	3	4	5	6
antes	x_0	x_1	x_2	x_3	x_4	-	-
após	x_5	x_1	x_2	x_0	x_4	-	x_3

Para as questões a seguir use a sequência de inteiros X , cujos valores representam chaves de registros.

$$X = (80, 13, 33, 5, 55, 42, 62, 18, 79, 6).$$

Tais questões consideram ainda as seguintes funções *hashing*. Em $h(x, i)$, o parâmetro i , $i = 0, 1, 2, \dots$, deve ser interpretado como a i -ésima sondagem para a chave x .

$$h(x, i) = (h_1(x) + i h_2(x)) \bmod 11 \quad h_1(x) = x \bmod 11 \quad h_2(x) = \max \left\{ 1, \left\lfloor \frac{x}{11} \right\rfloor \right\} \bmod 11$$

- Supondo que a sequência de chaves X foi inserida numa tabela *hash* usando o método sondagem linear e a função h_1 , pode-se afirmar que o número médio de acessos é maior que 1,4?
- Inserindo a sequência de chaves X numa tabela *hash* usando o método duplo *hash* e a função h , observa-se uma melhoria em número médio de acessos quando comparado ao método sondagem linear para o mesmo exemplo?
- Após inserção da sequência X nas questões anteriores, são necessários quantos acessos para se pesquisar a chave 110 na tabela *hash* pelo método duplo *hash* e pelo método sondagem linear?
- O esforço durante a inserção do n -ésimo registro numa tabala *hash* usando o método de Brent é $O(n^2)$?
- O esforço durante a inserção do n -ésimo registro numa tabala *hash* usando o método de árvore binária é $O(n^2)$?
- Qual o estado do endereço 4 inserção da sequência de chaves em X , em cada um destes métodos: duplo *hash*, de Brent ou da árvore binária?
- Para o exemplo em X , há vantagem no uso do método árvore binária em relação ao método de Brent?
- Para a configuração de *hash* perfeito para o exemplo X , para o primeiro nível de mapeamento, gerou-se a função $f(x) = (x \bmod 97) \bmod 10$, escolhendo-se f de uma classe universal de funções *hash*, considere $m = 10$ para o primeiro nível. Quantos endereços do primeiro nível não estão ocupados após a inserção dos valores em X ?
- Seja H um conjunto com 1000 funções *hash* e considere que cada $f \in H$ mapeia uniformemente valores de chaves de um conjunto U em endereços $0, 1, \dots, 499$. Suponha ainda que qualquer $f \in H$ pode ser escolhida aleatoriamente de H . Sabe-se que há apenas um par de chaves distintas $x, y \in U$ tal que existe duas funções $f \in H$ tal que $f(x) = f(y)$. Para todos os demais pares de chaves de U , não há colisão. Pode-se afirmar que H é uma classe universal de funções *hash*?
- Considere o algoritmo de busca binária modificado de tal forma que, a cada iteração, ao invés de se particionar a sequência pela metade, escolhe-se um particionamento aleatório da sequência, escolhendo-se uma posição entre o seu início e final de forma aleatória. Pode-se afirmar que, em média, este algoritmo tem comportamento assintótico tão bom quanto o pior caso da pesquisa binária tradicional?

Hashing Universal e Perfeito

1. O que é uma coleção universal de funções de hashing?
2. Qual a importância de usar uma coleção universal de funções no método de hashing perfeito de Cormen?
3. Considere a definição de $\mathcal{H}_{p,m}$, como descrito em sala, mas com o valor de $b = 0$ fixo. Neste caso, $\mathcal{H}_{p,m}$ é universal? Qual a probabilidade de se escolher uma função $h_{a,b} \in \mathcal{H}_{13,11}$ tal que $h_{a,b}(1) = h_{a,b}(12)$?
4. Deseja-se incluir $\{67, 79, 58, 89, 23, 22, 65\}$ usando o método *hash* perfeito em dois níveis visto em sala. Para o primeiro nível, a função $h(x) = ((2x + 3) \bmod 101) \bmod 7$ foi escolhida de um conjunto universal de funções *hash*. Mostre a configuração final dos dois níveis após incluir as chaves. Por simplificação, use a função $h_i(x) = x \bmod m_i$ a fim de incluir x no segundo nível, com m_i dependente do resultado associado ao endereço i no primeiro nível.
5. Um programa foi desenvolvido para fornecer *hashing* perfeito para as chaves em X , definido em (1), usando o método visto em sala. Para tanto, considerou-se

$$h_{a,b}(x) = ((a x + b) \bmod p) \bmod m$$

como possíveis funções de uma classe de funções *hashing* universal. Devido a um erro de implementação, os valores de a e b foram fixados em 1. Nestas condições, mostre como fica a disposição do arquivo após a inserção das chaves para $p = 101$. O programa implementado funciona corretamente?